

Technical & Security Report: Client-Side VTXO Verification in Arkade SDK

Project Title: Arkade SDK Client-Side Core

Author: Marcelo Adrián Guerra Najarro

Date: April 8, 2026

Subject: Plan B Network Technical Assignment — Technical Audit & Documentation

Repository Information

- **Private Repository:** <https://github.com/marchelo23/ln-markets-arkade-vtxo>
-

1. The Verification Algorithm & Security Properties

The implementation provides a **Zero-Trust** client-side verification pipeline. Given a VTXO outpoint received from the ASP, the SDK independently reconstructs and validates the full virtual transaction DAG without trusting any ASP-provided data.

1.1 DAG Reconstruction Algorithm

The core function `reconstructAndValidateVtxoDAG()` executes a multi-phase pipeline:

1. **Chain Discovery:** Fetches the VTXO chain from the `IndexerService`. Supports two modes: direct VTXO lookup (`getVtxoChain`) or privacy-preserving batch fetch (`getBatchVtxos`) where the client requests all chains in a commitment batch and filters locally.
2. **PSBT Parsing & TXID Verification:** Each virtual transaction PSBT is decoded using `@scure/btc-signer`. The computed `tx.id` is compared against the ASP-claimed `txid` — any mismatch throws `TXID_MISMATCH`.
3. **DAG Structural Validation:** For every node, `input[0]` must correctly reference the `txid:vout` of its ancestor. Value conservation is enforced: `sum(child outputs) == ancestor output[index]`. **Checkpoint transactions** are validated separately for sweep delay coherence and correct DAG integration.
4. **Cycle Detection (Ouroboros Protection):** Before wiring relationships, each node path is traced to detect cycles. A specially crafted DAG that loops back to itself would trigger an `OUROBOROS_CYCLE_DETECTED` exception rather than an infinite loop.
5. **Signature Verification:** Every node's `tapKeySig` is verified via BIP 340 Schnorr verification against the BIP 341 tweaked public key. The sighash is computed using `preimageWitnessV1()` with full previous output context. Only `SIGHASH_DEFAULT` (0x00) and `SIGHASH_ALL` (0x01) are accepted — non-standard flags are rejected.
6. **Taproot Script Tree Verification (Tier 2):** Merkle proofs are recomputed from tap leaf scripts and verified against the declared `tapMerkleRoot`. Ark exit policies are enforced: scripts must contain `CHECKSEQUENCEVERIFY + CHECKSIG` (standard exit), `HASH160 + CHECKSIG` (swap claim), or `CHECKLOCKTIMEVERIFY + CHECKSIG` (swap refund).

7. **Timelock Verification** (*Tier 2*): All nLockTime, nSequence, OP_CSV, and OP_CLTV constraints are extracted and validated for internal consistency and satisfiability against the current blockchain state.
8. **Hash Preimage Verification** (*Tier 2*): Scripts containing OP_SHA256, OP_HASH160, OP_HASH256, or OP_RIPEMD160 conditions are verified against provided preimages. This is demonstrated on a Boltz submarine swap (Ark ↔ LN) scenario where HASH160(preimage) satisfies the HTLC claim leaf.
9. **On-chain Anchoring** (*Tier 1*): The commitment transaction is fetched from a user-controlled Bitcoin node. The SDK verifies: (a) the transaction exists and is confirmed with sufficient depth, (b) the referenced output matches the expected script and amount, (c) confirmation depth satisfies the minimum threshold.

1.2 Security Properties

Property	Implementation
Zero Trust	Every ASP response is independently verified
Anti-Mirage	Raw tx hex is fetched and decoded; RPC responses validated
Cycle Protection	DFS-based cycle detection before relationship wiring
Sighash Shield	Only 0x00/0x01 accepted; 0x02, 0x81, etc. rejected
Iterative Traversal	Stack-based iteration prevents stack overflow on deep DAGs
Orphan Detection	Nodes unreachable from the anchoring leaf are rejected

2. Design Decisions and Trade-offs

2.1 Iterative vs. Recursive Traversal

All DAG traversals (signature verification, timelock checks, hash preimage verification) use iterative stack-based approaches. This prevents `RangeError: Maximum call stack size exceeded` on deep chains (tested up to depth=500). The trade-off is slightly more complex code, but guarantees deterministic resource usage.

2.2 Cryptographic Choices

- **AES-256-GCM** for exit data encryption: Provides authenticated encryption, preventing ciphertext manipulation attacks on stored exit transactions.
- **PBKDF2-HMAC-SHA256** (100,000 iterations): Derives the storage encryption key from user entropy with brute-force resistance.
- **Native node: crypto**: Used exclusively for the storage layer to minimize supply chain attack surface.

2.3 Privacy-Preserving Fetch

The SDK supports batch-wide VTXO fetching where clients request all chains in a commitment batch rather than a specific outpoint. This prevents the ASP from learning which VTXO the client is verifying. Trade-off: increased bandwidth consumption.

2.4 Dependencies

Runtime dependencies are limited to the standard Arkade SDK stack: @scure/btc-signer, @noble/curves, @noble/hashes, and @scure/base. No additional third-party libraries are introduced.

3. Completed Tiers & Limitations

3.1 Tier 1: Core VTXO Chain Verification

- **DAG Reconstruction:** Fully implemented with iterative traversal, cycle detection, and orphan rejection.
- **Signature Verification:** BIP 340 Schnorr verification with BIP 341 sighash computation and tweaked key validation.
- **Checkpoint Verification:** Structural coherence and sweep delay validation.
- **On-chain Anchoring:** Commitment confirmation, output script/amount matching, and depth verification via Bitcoin Core RPC or Electrum.

3.2 Tier 2: Full Script Satisfaction

- **Taproot Script Trees:** Merkle proof verification and Ark exit policy enforcement.
- **Timelocks:** Comprehensive nLockTime, nSequence, CSV, and CLTV validation.
- **Hash Preimages:** SHA256, HASH160, HASH256, RIPEMD160 preimage verification. Demonstrated on a Boltz submarine swap HTLC with claim and refund script leaves.

3.3 Tier 3: Sovereign Unilateral Exit

- **Data Identification:** `extractExitSequence()` traces the full path from anchoring leaf to VTXO root.
- **Local Storage:** `persistVtxoForExit()` encrypts and stores exit data via the SDK's storage adapter. `onReceiveVtxo()` automates this on VTXO receipt.
- **Exit Execution:** `executeSovereignExit()` broadcasts the full transaction sequence using only local data and a Bitcoin node — no ASP contact required.

3.4 Tier 4: Sentinel Protocol Frontend (Verification UI)

- **Purpose:** A tactical React-based interface ("Sentinel Protocol") designed to visually demonstrate the Arkade SDK's VTXO verification pipeline.
- **Branding:** Integrates the "CVE (Chelo Verification Engine)" branding, serving as a robust, high-fidelity proof of the SDK's security and logic.
- **Core Views:**
 - **Command Center:** Overall status dashboard showing pipeline health, verification stats, and anchoring status.
 - **DAG Explorer:** Structural analysis and visualization of the reconstructed VTXO dependency graph.
 - **Signature Audit:** Signature Verification Audit ensuring cryptographic integrity of the paths.
 - **Sovereign Exit:** Visualizes the exit sequence and cryptographic guarantees for unilateral extraction.
 - **Live Terminal:** Real-time diagnostic terminal representing the verification pipeline log output.

3.5 Limitations

- **MuSig2 Key Aggregation:** The SDK verifies that signatures are valid against tweaked public keys but does not independently verify the n-of-n MuSig2 key aggregation ceremony. This would require access to individual signer public keys, which are not exposed by the current IndexerService API.
 - **Regtest/Signet Scope:** Tested against regtest and signet structures. Mainnet deployment would benefit from BIP 157/158 (Neutrino) integration for trust-minimized on-chain verification.
 - **Integration Testing:** The integration test against a live arkd instance requires a running local environment. Without arkd, the test skips gracefully.
-

4. Test Summary

Suite	Tests	Description
vtxDAGVerification	13	Core DAG + signatures + Boltz HTLC
blackboxSec	21	Anti-Mirage, RPC validation, tamper detection, economic inflation, cycles, and orphan payloads
sovereignStorage	17	AES-256-GCM encryption, exit sequence, unilateral exit
stress_dos	16	Deep chains, concurrency, fail-fast
extreme_fuzzing	6	Garbage injection, sighash enforcement
real_world_vtxo	15	Multi-depth chain, checkpoint verification
stress	15	500-depth iterative traversal, concurrency
arkd_integration	1	E2E against local arkd or built-in Node.js Mock Server
Total	104	

Status: All 104 tests pass. Integration testing includes a fully functional native Node.js Mock Server to emulate Arkd Indexer and Bitcoin Core if live instances are unavailable.